

DesignSystemsJS - Building a Design Systems API for aiding standardization and AI integration

Hardik Shah
Department of Information Technology
Rochester Institute of Technology
USA
hds6825@rit.edu

Navin Kamuni
Department of AI-ML
BITS Pilani WILP
USA
navin.kamuni@gmail.com

Abstract—In the rapidly evolving landscape of software development, Design Systems have emerged as crucial frameworks that enforce consistency and efficiency in user interface (UI) and user experience (UX) design. However, the implementation of these systems varies significantly across teams, leading to discrepancies in understanding and applying design requirements. This inconsistency often results in divergent costs, efforts, and timelines in software projects. Our research introduces "DesignSystemsJS," a novel JavaScript library, to address these challenges. Leveraging the widespread popularity and flexibility of JavaScript, widely recognized as a dominant programming language in both client and server-side web applications, DesignSystemsJS aims to standardize Design Systems requirements. It does so by enforcing a JSON format specification for defining the Design System, based on a meticulously researched checklist. This standardization is crucial in a domain where software reuse and the evolution of JavaScript applications often pose challenges related to third-party dependencies. Furthermore, DesignSystemsJS aims to support Artificial Intelligence (AI) integration, reflecting the ongoing paradigm shift in the software development industry towards generative AI assistants. AI integration in software testing and development has seen significant advancements, offering various approaches and tools to enhance efficiency and effectiveness. Our library taps into these advancements, utilizing AI techniques like data mining and machine learning to promote automated software reuse, a key aspect of modern software development. DesignSystemsJS's architecture and API facilitate a more unified and streamlined approach to designing and implementing Design Systems, potentially revolutionizing how design consistency is maintained across different platforms and tools. This research paper details the methodology behind DesignSystemsJS, its features, and the potential impact it holds for the future of Design Systems development and AI integration.

Keywords—DesignSystemsJS, Design Systems, Design Systems API, Design Systems AI integration

I. INTRODUCTION

In the rapidly evolving landscape of software development, Design Systems have emerged as a cornerstone for ensuring consistency and efficiency [1]. These systems offer a comprehensive set of standards, principles, and components that guide the design process across various platforms and applications. The implementation of Design Systems has proven to significantly enhance user experience, streamline development processes, and foster a more cohesive brand identity [2]. However, the benefits of Design Systems are often undermined by the challenges associated with their standardization and integration [3].

The issue lies in the subjective interpretation and implementation of these systems. Different teams and

organizations perceive and apply Design System requirements in varying ways, leading to inconsistencies and inefficiencies [1]. Such discrepancies not only impact the uniformity and quality of the design output but also contribute to escalated costs, increased effort, and extended timelines in the development process [4].

Addressing these challenges necessitates a solution that can standardize the requirements and implementation of Design Systems. The proposed approach involves the development of an Application Programming Interface (API) that leverages Object-Oriented programming to integrate and standardize different Design Systems [2]. This paper introduces DesignSystemsJS, a JavaScript library that provides a structured and consistent framework for aiding standardization and AI integration [5]. By enforcing a JSON-based specification derived from a well-researched checklist, DesignSystemsJS aims to align the diverse interpretations and implementations of Design Systems, thereby mitigating the associated challenges [4].

Furthermore, the integration of Artificial Intelligence (AI) within this framework opens new avenues for enhancing Design System capabilities [1]. AI can provide intelligent insights, automate repetitive tasks, and facilitate adaptive design decisions, thereby enriching the design process and outcome [2]. The convergence of Design Systems with AI through an API presents a novel approach in the realm of software design, promising to revolutionize how design standards are set, maintained, and evolved [3].

This paper seeks to explore the development, features, and potential impact of DesignSystemsJS, focusing on its role in standardizing Design System requirements and integrating AI functionalities. The paper also delves into the methodology, underlying technologies, and the architectural framework of DesignSystemsJS, providing insights into its practical applications and prospects in software development.

II. IMPORTANCE OF DESIGN SYSTEMS

Design Systems play a crucial role in the software development lifecycle. They are not only pivotal in ensuring consistency and efficiency across projects but also have a significant impact on software quality, reliability, and maintainability. Below is an elaboration on the importance of Design Systems in software development.

1. *Software Design Methodology*: A well-structured software design methodology is essential for software development. It supports software reuse, promoting both software quality and reliability. Methodologies like the waterfall model guide the natural progression of development

steps, ensuring a structured approach to project management, cost-benefit analysis, and documentation [6][7].

2. *Design Decisions in Software Engineering*: In the realm of software engineering, design decisions are fundamental. They shape the fundamentals of software development and have a lasting impact on the final product. Implementing methodologies that balance fundamental knowledge with applied skills in real-world projects is critical. This approach, coupled with Just-In-Time (JIT) teaching, helps in developing scalable knowledge essential for making informed design decisions [8].

3. *Impact of Design Patterns on Software Quality*: Software engineers often rely on design patterns, which are repeated solutions to common problems in software design. These patterns, introduced in architecture by Alexander in 1974 and later adapted to software development by Gamma et al., have been instrumental in improving the design process. Design patterns contribute significantly to the quality of software by providing a tried-and-tested framework for addressing recurring design challenges [9].

4. *Maintenance and Reusability*: The selection of appropriate design patterns during the initial phases of design and development is critical to avoid monotonous and expensive maintenance later. Systems built with the right design patterns ensure easier maintenance and enhance the reusability of the code. This approach leads to a more sustainable and cost-effective development process [10].

5. *Design Pattern Recommendations*: Design patterns are essential for the development process as they provide reusability, modularization, quality, and consistency between design and implementation. The selection of appropriate design patterns requires expert knowledge. The recommendation and application of these patterns are important tasks in software development, as they significantly influence the overall quality and maintainability of the software product [11].

In summary, the importance of Design Systems in software development lies in their ability to guide design decisions, ensure consistency and quality, promote reusability and maintainability, and provide a framework for efficient and effective software development.

III. CHALLENGES WITH DESIGN SYSTEMS

1. *Inconsistent Interpretation of Design System Requirements*: One of the primary challenges in the implementation of Design Systems is the inconsistent interpretation of their requirements across different teams. Design Systems, by their nature, are meant to provide a unified and coherent set of guidelines and components for product development. However, in practice, the interpretation of what constitutes these systems can vary significantly [12]. For example, while one team might prioritize UI components and style guides, another might focus on interaction patterns or coding standards. This inconsistency often leads to a lack of cohesion in the user experience across different parts of the same product or different products within the same ecosystem [13].

2. *Divergent Implementation Strategies*: Closely related to the problem of inconsistent interpretation is the issue of divergent implementation strategies. Different teams may adopt varied approaches to integrate and utilize Design Systems in their workflows [14]. While some might fully embed these systems into their development processes, others may use them more as a reference point, leading to discrepancies in the final product. This divergence not only affects the consistency of the user experience but can also lead to increased maintenance costs and challenges in scaling the Design Systems as the product evolves [15].

3. *Variability in Cost, Effort, and Time*: The third significant challenge is the variability in cost, effort, and time resulting from the perceived requirements and chosen implementation approach. Teams that interpret Design System requirements broadly or choose to implement them in a comprehensive manner might incur higher initial costs and longer development times [13]. Conversely, a more limited interpretation or implementation might reduce costs and development time in the short term but can lead to increased technical debt, reduced scalability, and higher long-term maintenance costs [12]. This variability poses a significant challenge for organizations in planning resources and timelines for product development.

4. *The Need for Standardization*: These underlying problems highlight the need for a standardized approach to Design Systems [14]. Without standardization, the benefits of Design Systems – such as improved efficiency, consistency, and scalability – can be significantly undermined [15]. Standardization can also aid in better resource allocation, more predictable development timelines, and overall improved product quality.

The next section of this paper will discuss a proposed solution to these challenges through the development of DesignSystemsJS, a JavaScript library designed to standardize the requirements and implementation of Design Systems.

IV. SOLUTION

Building a Design System based on a well-researched Design Systems checklist can vastly assist in planning and building a Design System. The standardization of Application Programming Interfaces (APIs) plays a crucial role in this endeavor [16].

To standardize the requirements for Design Systems, one approach is to build a library using Object-Oriented programming. This library can integrate different Design Systems built on a checklist and can offer an API for extending applications. The approach towards a standard API design, as demonstrated in the context of open services in smart buildings, is relevant here. Such a design can handle a wide variety of devices by designing each device API to have two endpoints: attribute and state, based on device abstraction [17]. This approach can be adapted for Design Systems, ensuring flexible yet standardized communication between different components.

DesignSystemsJS, a JavaScript library, is an attempt to standardize requirements for a Design System. This library enforces Design System makers to define Design System specifications via JSON format based on a checklist. It offers an API which aims to establish a standardized method for accessing and manipulating Design System components,

thereby enhancing consistency across various platforms and tools. The development of a REST-API-based Service Oriented Architecture, as discussed in related work on industrial service architectures, provides a valuable reference for constructing efficient, scalable, and interoperable APIs for Design Systems [18].

Furthermore, the conformance assessment of API designs in relation to API design decisions is a critical aspect of standardization. A method for automated conformance assessment of API designs ensures that the APIs adhere to established patterns and practices, enhancing the reliability and consistency of the Design Systems [19].

In summary, the development of DesignSystemsJS, influenced by existing standards and practices in API design and standardization, represents a significant step towards achieving a unified and efficient approach to Design System implementation. This initiative addresses the varying perceptions and implementations of Design Systems, aiming to reduce the associated costs, efforts, and time discrepancies.

V. FEATURES SUPPORTED IN DESIGNSYSTEMSJS

1. *Standardized Method for Design System components:* DesignSystemsJS aims to establish a standardized method for creating, accessing, and manipulating Design System parts. This approach is akin to the role of APIs in software development, facilitating cooperation between programs despite differences in hardware, language, or architecture. The API model becomes a perfect match for communication between designers and developers, enhancing the efficiency and consistency of development processes across various platforms and tools.

2. *Adding Multiple Design Systems to the Library:* The ability to add multiple Design Systems to the library is crucial. Design systems, as abstractions of the design process, provide essential documentation for developers, streamlining the application development process. For instance, Salesforce's Lightning Design System demonstrates how specific markup ensures consistent user experiences across platforms, a principle that can be leveraged by DesignSystemsJS.

3. *Active Design System Color Application:* Once a Design System is activated, DesignSystemsJS applies colors to the root HTML element. This feature addresses a common need in design systems: the easy application and modification of aesthetic elements like colors. For example, Salesforce's Lightning Design System requires a dedicated team for maintenance due to its low-level abstraction nature. In contrast, DesignSystemsJS simplifies this process by automating color application upon activation, streamlining the design integration process.

4. *Design Tokens and Standard Formats:* DesignSystemsJS leverages the concept of design tokens and standard formats. Design tokens, like those used in Salesforce's Lightning or Shopify's Polaris, are structured values in a machine-readable file, simplifying the design integration process. The emergence of standard formats enables seamless working with multiple design systems and easier migration between them, reducing the dependency on consistent naming schemes.

5. *Future of Design APIs and Networked Design Systems:* The future of Design APIs points towards networked design systems, where sets of applications and tools can

communicate about design decisions, vastly improving the scalability and adaptability of design integration.

6. *Automating Design Concerns and Accessibility:* DesignSystemsJS can automate design concerns like accessibility, running tests and validating guidelines in line with standards like WCAG. This feature ensures that design decisions adhere to accessibility standards, enhancing the user experience across different platforms [20].

7. *Leveraging Design APIs for Large-Scale Maintenance:* The use of a design API like DesignSystemsJS enables the maintenance of large and complex codebases, ensuring quality and consistency without sacrificing the agility needed in modern software development [21].

In summary, DesignSystemsJS offers a comprehensive suite of features that standardize and streamline the integration of design systems into software development, addressing the challenges of consistency, scalability, and accessibility in a rapidly evolving digital landscape.

VI. IMPLEMENTATION

A. Underlying Technologies

1. *DesignSystemsJS Library:* Written in TypeScript, a superset of JavaScript, enhances JavaScript's capabilities by introducing strong typing and object-oriented programming principles. TypeScript compiles to JavaScript, ensuring compatibility with web standards and existing JavaScript frameworks [22].

2. *Client-side integration:* Namely HTML5, CSS3, JavaScript, JSON. Client-side technologies play a crucial role in web development. HTML5 and CSS3 provide the structural foundation and styling of web pages. HTML5's capabilities are enhanced by CSS3, offering more dynamic and responsive designs [23][24]. JavaScript, as a dynamic scripting language, provides interactivity and functionality to web applications. It is widely used in both client and server-side applications, demonstrating its versatility and importance in modern web development [25]. JSON, a lightweight data-interchange format, complements these technologies by facilitating data exchange between the server and web applications [26].

3. *How the API works:* The API, implemented in JavaScript, provides a standardized interface for accessing and manipulating design system components.

- *Consume DesignSystemsJS library and import the DesignSystemsJS class:* As illustrated in [5, Fig. 1], the DesignSystemsJS class encapsulates the functionalities and properties of the Design System, making it accessible through a well-defined API.
- *Pass DesignSystem specifications in JSON file adhering to DesignSystemsJS JSON specification:* As illustrated in [5, Fig. 2], specifications for Design Systems are passed as JSON files, adhering to a predefined schema, ensuring standardization and consistency across different implementations. The color values from the JSON file are mapped to CSS3 variables in the styles file, as illustrated in [5, Fig. 3].
- *Setting the root element:* As illustrated in [5, Fig. 4], DesignSystemsJS dynamically applies color and styling from SampleDesignSystem to the root HTML element, demonstrating the integration of

CSS3 and JavaScript in real-time customization of web interfaces.

```
src > index.html > {} "index.html" > html > head > script
1 <!doctype html>
2 <html lang="en">
3 <head>
4 <title>DesignSystemsJS Test</title>
5
6 <link rel="stylesheet" href="/webpage_assets/css/styles.css" />
7
8 <script type="module">
9   import DesignSystemsJS from './index.js';
10  const { default: sampleDesignSystem } = await import("./data/SampleDesignSystem.json",
11    { assert: { type: "json" } });
12
13  // Initialize DesignSystemsJS. Pass sampleDesignSystem JSON.
14  const DesignSystems = new DesignSystemsJS([sampleDesignSystem]);
15  DesignSystems.setRootElement(document.documentElement);
16
17  // Set the DesignSystems object in global scope
18  window.DesignSystems = DesignSystems;
19 </script>
20 </head>
21 <body>
22 <h1 class="sds-text-primary">DesignSystemsJS Test</h1>
23
24 <form name="formTest" method="GET" onsubmit="return false;">
25   <div>
26     <label for="fullName" class="sds-label">Full name</label>;
27     <input id="fullName" name="fullName" type="text" class="sds-textfield" />
28   </div>
29   <div>
30     <label for="sex" class="sds-label">Sex</label>;
31     <input id="sex" name="sex" type="radio" value="M" class="sds-radio">Male</input>&nbsp;
32     <input id="sex" name="sex" type="radio" value="F" class="sds-radio">Female</input>
33   </div>
34   <div style="padding-top: 20px;">
35     <input type="submit" text="Submit" class="sds-button-primary" />
36   </div>
37 </form>
38 </body>
39 </html>
```

Fig. 1. DesignSystemsJS - Example - Setting SampleDesignSystem object, Source [5]

```
src > data > {} SampleDesignSystem.json > {} foundations > {} colors
1 {
2   "id": "sample-design-system",
3   "name": "Sample Design System",
4   "description": "Use Sample's end-to-end design language to create simple,
5     intuitive and beautiful experiences.",
6   "url": "https://no-url-available.com/",
7   "designLanguage": {},
8   "foundations": {
9     "colors": {
10       "text": {
11         "primary": "#172B4D",
12         "secondary": "#FFFFFF",
13         "tertiary": "string"
14       },
15       "background": {
16         "primary": "#FFFFFF",
17         "secondary": "#F4F5F7",
18         "tertiary": "string",
19         "success": "",
20         "information": "",
21         "warning": "",
22         "error": ""
23       },
24       "components": {
25         "primary": "#0052CC",
26         "secondary": "#8993A5",
27         "tertiary": "string"
28       },
29       "states": {
30         "default": "",
31         "hover": "#0065FF",
32         "pressed": "#0747A6"
33       }
34     },
35     "layout": {},
36     "typography": {},
37     "elevation": {},
38     "motion": {},
39     "iconography": {}
40   },
41   "components": {},
42   "maintenance": {}
43 }
```

Fig. 2. DesignSystemsJS - Example - SampleDesignSystem's JSON object, Source [5]

```
src > webpage_assets > css > # styles.css > ...
1 /* Styles for page header */
2 .sds-text-primary {
3   color: var(--text-primary);
4 }
5
6 /* Styles for label component */
7 .sds-label {
8   color: var(--text-primary);
9   background-color: var(--background-primary);
10 }
11
12 /* Styles for textfield component */
13 .sds-textfield {
14   color: var(--text-primary);
15   background-color: var(--background-primary);
16 }
17 .sds-textfield:hover {
18   color: var(--text-primary);
19   background-color: var(--background-secondary);
20 }
21 .sds-textfield:active {
22   color: var(--text-primary);
23   background-color: var(--background-primary);
24 }
25
```

Fig. 3. DesignSystemsJS - Example - CSS3 styles file, Source [5]



Fig. 4. DesignSystemsJS - Example - Browser Preview, Source [5]

B. Architecture of DesignSystemsJS Library

- *TypeScript Type definitions*: Offers interfaces for Design System components. The TypeScript type definitions as illustrated in [5, Fig. 5] provide a structured framework for defining the properties and behaviors of Design System components, ensuring a standardized approach to design system.
- *TypeScript classes for library*: This includes the DesignSystem and DesignSystems class as illustrated in [5, Fig. 6] which implement their respective type definitions. These classes form the backbone of the library, providing the

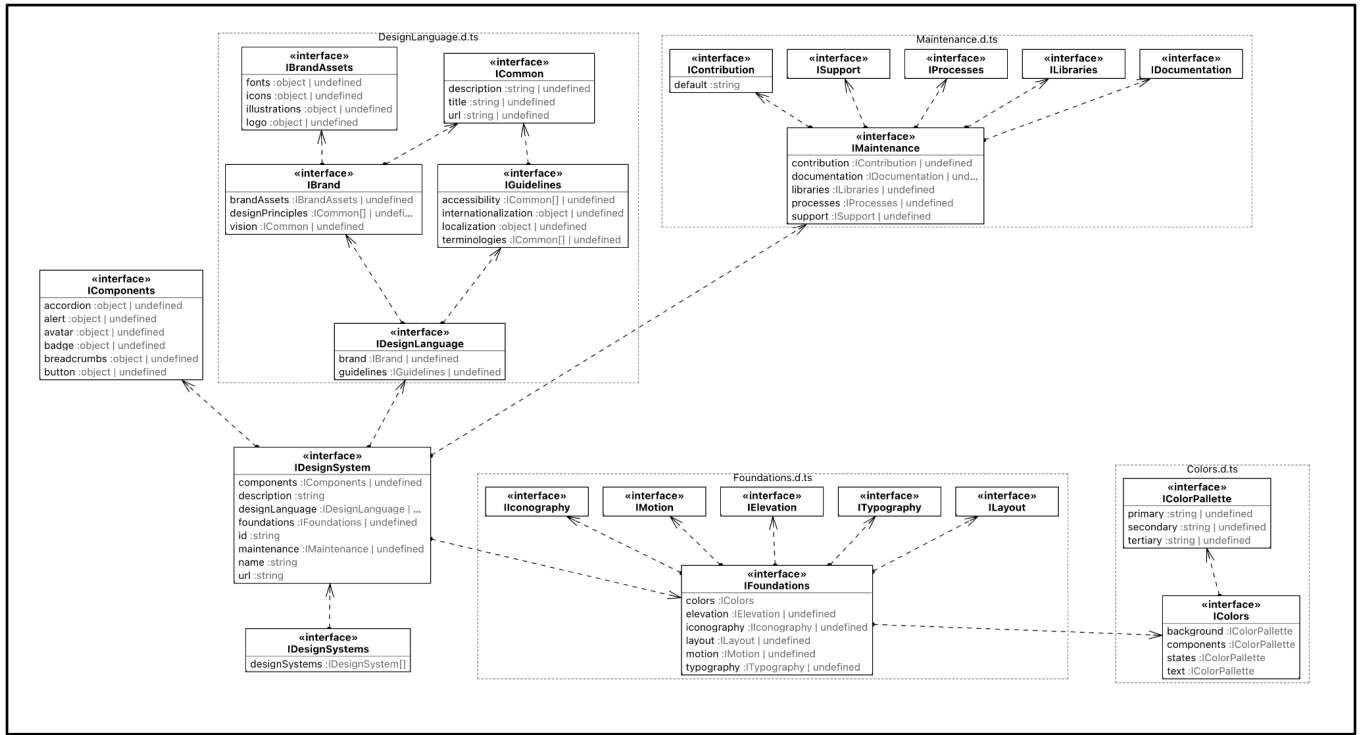


Fig. 5. DesignSystemsJS - Types - Class Diagram, Source [5]

functionalities required for managing and manipulating the current Design System.

- *TypeScript entry class which exports the DesignSystems class*: The entry class serves as the interface for the library, exporting the DesignSystems class, which is the core of the library.



Fig. 6. DesignSystemsJS - Classes - Class Diagram, Source [5]

- *Written in TypeScript and compiled to a JavaScript file for consumption on client-side*: The library is developed using TypeScript offering advantages such as type safety and class-based object-oriented programming. It is then compiled to JavaScript for broad compatibility with web technologies including HTML5. The Design System object should be a JSON and should adhere to the IDesignSystem interface.

VII. AI INTEGRATION WITH DESIGNSYSTEMSJS

The integration of Artificial Intelligence (AI) into Design Systems, as exemplified by DesignSystemsJS, represents a significant advancement in the field of software development and design. AI technologies, including machine learning and data analytics, can be leveraged to enhance the functionality, efficiency, and adaptability of Design Systems. This integration promises not only to streamline the design process but also to introduce a new level of intelligence in design decision-making.

AI in software testing has been recognized as a crucial evolution, especially given the increasing complexity of software systems where traditional manual testing methods are becoming less feasible [27]. In the context of DesignSystemsJS, AI could be employed to automate and refine the testing of design components, ensuring higher quality and consistency. For instance, machine learning algorithms could analyze user interactions with the design system to identify patterns and anomalies, leading to more intuitive and user-friendly designs.

The development and integration of AI in the software development lifecycle, especially in planning and architecture stages, are gaining importance [2]. In DesignSystemsJS, AI can play a role in the planning phase by analyzing existing design patterns and suggesting optimizations or new design elements based on current trends and best practices. This would not only save time but also ensure that the design systems are up-to-date and competitive.

Machine learning, a subset of AI, has become integral in the design of intelligent systems across various disciplines. It is crucial to evaluate how systems and software engineering approaches can effectively integrate with machine learning to enhance the intelligence of software systems [28]. Within DesignSystemsJS, machine learning can be used to adapt design systems based on user feedback and interaction data.

This could lead to dynamically evolving design systems that continually improve and adapt to user needs.

Moreover, the integration of AI with database management systems (DBMS) has shown potential in shaping the future of computing [29]. For DesignSystemsJS, this integration could mean the efficient management of design elements and their metadata, facilitating quicker retrieval, modification, and deployment of design components. AI-driven analytics could provide insights into the usage and performance of various design elements, enabling data-driven decision-making in design system evolution.

In conclusion, the integration of AI into DesignSystemsJS holds significant potential in revolutionizing the way design systems are developed, managed, and utilized. By leveraging AI, DesignSystemsJS can offer more dynamic, intelligent, and user-centric design solutions, thereby enhancing the overall efficiency and effectiveness of design systems in software development.

VIII. CONCLUSION

In this paper, we have explored the critical role of Design Systems in modern software development, emphasizing the challenges associated with their varied interpretation and implementation. Our focus on the development of DesignSystemsJS, a comprehensive JavaScript library, addresses these challenges by standardizing Design System requirements and integrating Artificial Intelligence functionalities.

DesignSystemsJS offers a unique solution to the fragmentation in Design Systems implementation. By using Object-Oriented programming principles to create a standardized API, DesignSystemsJS allows for a more unified approach. This standardization not only reduces variability in cost, effort, and time associated with Design Systems development but also enhances the overall quality and consistency of software design across different teams and platforms.

Furthermore, the integration of AI in DesignSystemsJS marks a significant advancement in the field of software design. AI capabilities embedded within the API enable intelligent automation, predictive analysis, and more dynamic, adaptive Design Systems. This integration is a promising step towards more efficient and effective design processes, potentially revolutionizing how Design Systems are conceived and executed.

As we move forward, the potential for further advancements in DesignSystemsJS is vast. Future work could explore deeper AI integrations, such as using machine learning algorithms for automated design decision-making or natural language processing for more intuitive design interfaces. The ongoing development of DesignSystemsJS is poised to not only address current challenges in Design Systems but also to set new standards in software design excellence.

REFERENCES

- [1] L. M. Amugongo, A. Kriebitz, A. Boch, and C. Lütge, "Operationalising AI ethics through the agile software development lifecycle: a case study of AI-enabled mobile health applications," *AI Ethics*, Aug. 2023, doi: [10.1007/s43681-023-00331-3](https://doi.org/10.1007/s43681-023-00331-3).
- [2] I. Georgievski, "Conceptualising Software Development Lifecycle for Engineering AI Planning Systems," in *2023 IEEE/ACM 2nd International Conference on AI Engineering – Software Engineering for AI (CAIN)*, May 2023, pp. 88–89. doi: [10.1109/CAIN58948.2023.00019](https://doi.org/10.1109/CAIN58948.2023.00019).
- [3] A. Nurwidyantoro *et al.*, "Integrating human values in software development using a human values dashboard," *Empir Software Eng.*, vol. 28, no. 3, p. 67, Apr. 2023, doi: [10.1007/s10664-023-10305-y](https://doi.org/10.1007/s10664-023-10305-y).
- [4] M. L. Varshika, A. K. Mishra, N. Kandasamy, and A. Das, "Hardware-Software Co-Design for On-Chip Learning in AI Systems," in *Proceedings of the 28th Asia and South Pacific Design Automation Conference*, in ASPDAC '23. New York, NY, USA: Association for Computing Machinery, Jan. 2023, pp. 624–631. doi: [10.1145/3566097.3568359](https://doi.org/10.1145/3566097.3568359).
- [5] Hardik Shah, "gameboyzone/DesignSystemsJS: Building a JavaScript API for Design Systems facilitating Standardization and AI Integration," DesignSystemsJS - GitHub source code repo. Accessed: Dec. 04, 2023. Available: <https://github.com/gameboyzone/DesignSystemsJS>
- [6] A. Miller, "Engineering design: its importance for software," *IEEE Potentials*, vol. 8, no. 2, pp. 14–16, May 1989, doi: [10.1109/45.31589](https://doi.org/10.1109/45.31589).
- [7] N. Raeesinejad, M. Moshirpour, L. Behjat, and Y. Afshar, "Design Decisions Matter: Conveying the Importance of Software Engineering Best Practices through Hybrid PBL," in *2022 IEEE Frontiers in Education Conference (FIE)*, Oct. 2022, pp. 1–9. doi: [10.1109/FIE56618.2022.9962418](https://doi.org/10.1109/FIE56618.2022.9962418).
- [8] S. A. El-Seoud and M. M. El-Khouly, "Information system design methodologies: software development and methods of integration," in *Proceedings. 2004 International Conference on Information and Communication Technologies: From Theory to Applications, 2004.*, Apr. 2004, pp. 573–574. doi: [10.1109/ICTTA.2004.1307892](https://doi.org/10.1109/ICTTA.2004.1307892).
- [9] F. Khomh and Y.-G. Guéhéneuc, "Design patterns impact on software quality: Where are the theories?," in *2018 IEEE 25th International Conference on Software Analysis, Evolution and Reengineering (SANER)*, Mar. 2018, pp. 15–25. doi: [10.1109/SANER.2018.8330193](https://doi.org/10.1109/SANER.2018.8330193).
- [10] F. Palma, H. Farzin, Y.-G. Guéhéneuc, and N. Moha, "Recommendation system for design patterns in software development: An DPR overview," in *2012 Third International Workshop on Recommendation Systems for Software Engineering (RSSE)*, Jun. 2012, pp. 1–5. doi: [10.1109/RSSE.2012.6233399](https://doi.org/10.1109/RSSE.2012.6233399).
- [11] M. Z. Asghar, K. A. Alam, and S. Javed, "Software Design Patterns Recommendation : A Systematic Literature Review," in *2019 International Conference on Frontiers of Information Technology (FIT)*, Dec. 2019, pp. 167–1675. doi: [10.1109/FIT47737.2019.00040](https://doi.org/10.1109/FIT47737.2019.00040).
- [12] R. Jongeling, J. Fredriksson, F. Ciccozzi, A. Cicchetti, and J. Carlson, "Towards Consistency Checking Between a System Model and Its Implementation," in *Systems Modelling and Management*, Ö. Babur, J. Denil, and B. Vogel-Heuser, Eds., in Communications in Computer and Information Science. Cham: Springer International Publishing, 2020, pp. 30–39. doi: [10.1007/978-3-030-58167-1_3](https://doi.org/10.1007/978-3-030-58167-1_3).
- [13] J. L. Tekaati, H. Anacker, and R. Dumitrescu, "The Paradigm of Design Thinking and Systems Engineering in the Design of Cyber-Physical Systems: A Systematic Literature Review," in *2021 IEEE*

- International Symposium on Systems Engineering (ISSE)*, Sep. 2021, pp. 1–8. doi: [10.1109/ISSE51541.2021.9582548](https://doi.org/10.1109/ISSE51541.2021.9582548).
- [14] G. Ramesh, T. V. Rajini Kanth, and A. Ananda Rao, “Extensible Real Time Software Design Inconsistency Checker: A Model Driven Approach,” in *Proceedings of the International MultiConference of Engineers and Computer Scientists*, Hong Kong, Mar. 2016. Accessed: Dec. 05, 2023. Available: https://www.iaeng.org/publication/IMECS2016/IMECS2016_pp454-460.pdf
- [15] M. Zuefle, S. Muschik, N. Bursac, and D. Krause, “Coping Asynchronous Modular Product Design by Modelling a Systems-in-System,” *Proceedings of the Design Society*, vol. 2, pp. 2553–2562, May 2022, doi: [10.1017/pds.2022.258](https://doi.org/10.1017/pds.2022.258).
- [16] P. Troger, H. Rajic, A. Haas, and P. Domagalski, “Standardization of an API for Distributed Resource Management Systems,” in *Seventh IEEE International Symposium on Cluster Computing and the Grid (CCGrid '07)*, May 2007, pp. 619–626. doi: [10.1109/CCGRID.2007.109](https://doi.org/10.1109/CCGRID.2007.109).
- [17] S. Bandara, T. Yashiro, N. Koshizuka, and K. Sakamura, “Towards a standard API design for open services in smart buildings,” in *2016 TRON Symposium (TRONSHOW)*, Dec. 2016, pp. 1–7. doi: [10.1109/TRONSHOW.2016.7842883](https://doi.org/10.1109/TRONSHOW.2016.7842883).
- [18] M. S. Alam, U. D. Atmojo, J. O. Blech, and J. L. M. Lastra, “A REST and HTTP-based Service Architecture for Industrial Facilities,” in *2020 IEEE Conference on Industrial Cyberphysical Systems (ICPS)*, Jun. 2020, pp. 398–401. doi: [10.1109/ICPS48405.2020.9274792](https://doi.org/10.1109/ICPS48405.2020.9274792).
- [19] A. Singjai and U. Zdun, “API Description-Based Conformance Assessment of Architectural Design Decision,” in *2022 IEEE International Conference on Service-Oriented System Engineering (SOSE)*, Aug. 2022, pp. 59–68. doi: [10.1109/SOSE55356.2022.00013](https://doi.org/10.1109/SOSE55356.2022.00013).
- [20] H. Shah, “Advancing Web Accessibility -- A guide to transitioning Design Systems from WCAG 2.0 to WCAG 2.1,” in *Computer Science & IT Conference Proceedings*, in 22, vol. 13. Academy & Industry Research Collaboration Center, Nov. 2023, pp. 233–245. doi: [10.5121/csit.2023.132218](https://doi.org/10.5121/csit.2023.132218).
- [21] Matthew Ström, “Design APIs: The Evolution of Design Systems,” Matthew Ström, designer-leader. Accessed: Dec. 06, 2023. Available: <https://matthewstrom.com/writing/design-apis/>
- [22] J. D. Scarsbrook, M. Utting, and R. K. L. Ko, “TypeScript’s Evolution: An Analysis of Feature Adoption Over Time,” in *2023 IEEE/ACM 20th International Conference on Mining Software Repositories (MSR)*, May 2023, pp. 109–114. doi: [10.1109/MSR59073.2023.00027](https://doi.org/10.1109/MSR59073.2023.00027).
- [23] N. Li and B. Zhang, “The Design and Implementation of Responsive Web Page Based on HTML5 and CSS3,” in *2019 International Conference on Machine Learning, Big Data and Business Intelligence (MLBDBI)*, Nov. 2019, pp. 373–376. doi: [10.1109/MLBDBI48998.2019.00084](https://doi.org/10.1109/MLBDBI48998.2019.00084).
- [24] H. Shah, “Harnessing customized built-in elements -- Empowering Component-Based Software Engineering and Design Systems with HTML5 Web Components,” in *Computer Science & IT Conference Proceedings*, in 22, vol. 13. Academy & Industry Research Collaboration Center, Nov. 2023, pp. 247–259. doi: [10.5121/csit.2023.132219](https://doi.org/10.5121/csit.2023.132219).
- [24] H. Shah, “Harnessing customized built-in elements -- Empowering Component-Based Software Engineering and Design Systems with HTML5 Web Components,” in *Artificial Intelligence, Soft Computing and Applications*, Academy & Industry Research Collaboration Center, Nov. 2023, pp. 247–259. doi: [10.5121/csit.2023.132219](https://doi.org/10.5121/csit.2023.132219).
- [25] R. Nascimento, A. Hora, and E. Figueiredo, “Exploring API Deprecation Evolution in JavaScript,” in *2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, Mar. 2022, pp. 169–173. doi: [10.1109/SANER53432.2022.00031](https://doi.org/10.1109/SANER53432.2022.00031).
- [26] Z. Haitao, “Research on Mobile Web Front-end Design Based on HTML5 Technology,” in *2021 6th International Symposium on Computer and Information Processing Technology (ISCIPIT)*, Jun. 2021, pp. 290–293. doi: [10.1109/ISCIPIT53667.2021.00065](https://doi.org/10.1109/ISCIPIT53667.2021.00065).
- [27] M. Islam, F. Khan, S. Alam, and M. Hasan, “Artificial Intelligence in Software Testing: A Systematic Review,” Sep. 2023. doi: [10.1109/TENCON58879.2023.10322349](https://doi.org/10.1109/TENCON58879.2023.10322349).
- [28] J. Alharbi and S. Bhattacharyya, “Machine Learning with System/Software Engineering in Selection and Integration of Intelligent Algorithms,” in *2021 IEEE International Systems Conference (SysCon)*, Apr. 2021, pp. 1–7. doi: [10.1109/SysCon48628.2021.9447111](https://doi.org/10.1109/SysCon48628.2021.9447111).
- [29] N. Nihalani, S. Silakari, and M. Motwani, “Integration of Artificial Intelligence and Database Management System: An Inventive Approach for Intelligent Databases,” in *2009 First International Conference on Computational Intelligence, Communication Systems and Networks*, Jul. 2009, pp. 35–40. doi: [10.1109/CICSYN.2009.54](https://doi.org/10.1109/CICSYN.2009.54).